

Approach: Conversational SHL Assessment Recommender

Problem framing

The task is an *agent design* problem more than a search problem: take a user from a vague intent to a grounded shortlist of SHL Individual Test Solutions through dialogue, deciding each turn whether to **clarify, recommend, refine, compare, or refuse** — and never recommend anything outside the catalog. I optimized for the three scored axes: hard evals (schema, catalog-only, ≤ 8 turns), Mean Recall@10, and behavior-probe pass-rate.

Stack and why

- **FastAPI + Pydantic v2** — the response schema is non-negotiable, so I let Pydantic enforce it on the way out; a malformed body is structurally impossible.
- **Groq (llama-3.1-8b-instant)** — free tier with high throughput. The two LLM jobs here (route a turn; pick ids from a candidate list) are easy for an 8B model, and its higher RPM/TPM limits matter because the grader hits my key. The model is configurable (GROQ_MODEL) for a paid 70B key.
- **BM25 (rank-bm25) + optional embeddings** — the catalog is only 377 items, so lexical retrieval is fast and strong on exact skills (Java, SQL, Docker). Embeddings (MiniLM, fused via Reciprocal Rank Fusion) are optional and off by default to keep the deploy light (no torch); retrieval degrades gracefully to BM25-only. No FAISS — brute-force cosine over 377 vectors is instant.

Data

The scraped catalog (shl_product_catalog.json, 377 records) has unescaped control characters, so it's parsed with `strict=False`. A build step normalizes it to a stable `{id, name, url, test_type, description, ..., search_text}`. `test_type` is **derived** from the catalog's `keys` labels using the eight SHL codes (K, P, A, S, C, B, D, E), comma-joined for multi-type items.

Architecture: stateless, two-call agent

Every `POST /chat` rebuilds all state from the full message history.

1. **Understand** (LLM $\rightarrow$ JSON): routes the turn (`clarify | recommend | refuse`), flags scope/injection, and emits an `expanded_query` plus explicit `must_include / must_exclude` edits from the latest message.
2. **Retrieve** (recommend only): hybrid search over the catalog. Test-type and job-level signals are *soft boosts*, never hard filters, to protect recall. A curated core set (OPQ32r, Verify G+, GSA, Graduate Scenarios — the recurring default components in the traces) is always made available to the selector.
3. **Compose** (LLM $\rightarrow$ JSON): selects 1–10 catalog **ids** from the candidate list and writes the reply (including grounded comparisons).
4. **Ground + guard**: the server maps ids $\rightarrow$ catalog records, so `name/url / test_type` always come from the data — **a URL can never be hallucinated**.

Clarify/refuse use one LLM call; recommend uses two. Worst case ~ 2 calls, well inside the 30 s timeout.

Context engineering

- The retrieval query is the LLM's `expanded_query` **concatenated with the literal user text**, so exact terms (SQL, Docker, "SVAR US") are never paraphrased away while synonyms ("works with stakeholders" $\rightarrow$ "communication/influencing") are still added.
- Compose is told to be *inclusive* (pick every clearly relevant item, 5–8 typical) and to add standard battery defaults (personality = OPQ32r, cognitive = Verify G+, graduate SGT = Graduate Scenarios) unless declined — a convention learned directly from the 10 traces.

Guards (deterministic, not model-trusted)

Every externally graded behavior is enforced in Python: honored edits (`must_exclude` items are filtered from the final list even if the model re-includes them; `must_include` topics are ensured present); a clarify budget (≤ 2 questions) so the 8-turn cap is never exhausted; dropping any fabricated id; refusing off-topic/legal/injection with empty recommendations; and a never-500 fallback.

Evaluation

- **Replay harness** over the 10 traces computes Mean Recall@10 by resolving each labeled shortlist to catalog ids and comparing against the agent's final recommendations. It paces requests to respect the free-tier RPM limit.
- **Behavior probes**: refuse off-topic/legal/injection, no-recommend on a vague turn-1, honor add/drop edits, every URL $\in$ catalog, ≤ 8 turns.
- **24 unit tests**: schema enforcement, catalog + `test_type` mapping, retrieval sanity, and the guards (with a fake LLM, so they run offline).

What didn't work / what moved the needle

- **First measurement: Recall@10 = 0.34.** Diagnosing it, I split the failure into a retrieval ceiling (items not in the candidate pool) vs. composition (items available but not picked). The retrieval ceiling was 0.74, so both needed work.
- **Silent rate-limit fallback was the biggest hidden bug.** The 70B model's free tier 429'd under the harness's burst, and every failed turn silently dropped to a weak BM25 fallback — making prompt changes look like no-ops. Switching to 8b-instant, adding 429-aware backoff, trimming the compose payload (24 candidates, short descriptions), and making the compose-failure fallback reuse the already-retrieved candidates (instead of naive BM25) took the mean to **~0.59+** and made it robust under throttling — which matters because the grader shares the same key.
- **Inclusive composition + battery defaults** fixed the systematic OPQ32r/Verify G+ misses; merging literal user text into the retrieval query fixed dropped exact terms (SQL, Docker).
- **Hardest residual misses** are report/variant products (e.g. "OPQ Universal Competency Report 2.0") and expert leaps with no direct catalog match (the Rust role → Linux/Networking). These need either family-expansion of selected instruments or a reranker — noted as future work over correctness.

AI tools used

I used an AI-assisted coding tool to speed up boilerplate scaffolding, the evaluation harness, and iteration. All design decisions — the two-call routing, id-only grounding, the deterministic guards, and the rate-limit mitigations — are my own and are defensible end to end.